*Islamic University of Technology*

# Lab 13: Pointer Arithmetic

CSE 4108 : Structured Programming Lab

**Section 1B**

MIA, NFS, IAC, ASH

March 06, 2025

# 1 Lab Tasks

*"I am the compiler of my code,*
*Addressed is my body with pointer to my blood,*
*I have created over a thousand addresses,*
*Unknown to death,*
*Nor known to life,*
*Have withstood pain to create many pointers,*
*Yet, those will never hold anything,*
*So as I print, Unlimited Pointer Works"*

This is a lab on pointer arithmetic. When processing an array, you are **not allowed** to use any subscript operator. Only pointer to array and array name as pointer with dereferencing/indirection is allowed. However, you can use subscript operator while declaring an array since it is necessary.

## 1.1 Dot Product of Row and Column

The dot product of two equal-length sequences of numbers is the sum of the products of their corresponding elements. In the case of matrices, the dot product of a row and a column produces a scalar value.

Write a function in C that takes two pointers (one for a row and one for a column), the length of the row/column, and the total number of columns in the matrix of the column as input. Using pointer arithmetic, compute the dot product and return the result. The function declaration is as follows:

```
double dotProduct(double *row, double *col, int len, int numCols);
```

Since the column values are not stored contiguously in memory, the function must correctly compute their locations using the total number of columns. Design your input and output accordingly. Use pointer arithmetic instead of array subscript notation.

**[Hint]** *Declare a 2D array in main, extract a row and a column using pointers, and pass them to the function. To access elements of the column, use pointer arithmetic based on the number of columns. Print the result in main.*

## 1.2 Matrix Multiplication Using Dot Product

Matrix multiplication is performed by computing the dot product of corresponding row and column vectors. Given two matrices $A$ and $B$ with dimensions $m \times n$ and $n \times p$, their multiplication results in a matrix $C$ of dimensions $m \times p$.

Write a function in C that takes three pointers representing two input matrices and one output matrix, along with their dimensions, and performs matrix multiplication using pointer arithmetic. Use the dot product function from subsection 1.1. The function declaration is as follows:

```
void matrixMultiply(int m, int n, int p, double A[][n], double B[][p], double C[][p
    ] );
```

Ensure that no array subscript notation is used. All operations should be performed using pointer arithmetic.

**[Hint]** *Use nested loops to iterate over rows of $A$ and columns of $B$, computing dot products and storing results in $C$. Print the matrices in main.*

### 1.3 Find Unique Elements in an Array

Given an array of integers, find and store only the unique elements in another array while maintaining their original order.

Write a function in C that takes an input array, its length, and an output array, and stores only the unique elements using pointer arithmetic. The function declaration is:

```
void findUniqueElements(int *arr, int *uniqueArr, int len, int *uniqueLen);
```

The function should update ‘uniqueLen‘ with the count of unique elements. Avoid using array subscript notation.

**[Hint]** *Use nested loops and pointer arithmetic to check for duplicates before adding elements to ‘uniqueArr‘. Print both arrays in the main function.*